

Naive Bayes

Supervised machine learning algorithm used for classification tasks.
Goal: model the posterior probability of a binary outcome (0 to 1)

Algorithm Flow:

- 1) Convert raw text into numerical representation (e.g., CountVectorizer, TF-IDF).
- 2) Calculate Prior Probabilities $P(\text{class})$ for each category.
- 3) Calculate Conditional Likelihoods $P(\text{word}|\text{class})$ using Laplacian smoothing
- 4) For a new input, compute the Posterior Probability for each class using Bayes' Theorem: $P(\text{class}|\text{features}) \propto P(\text{features}|\text{class}) * P(\text{class})$.
- 5) Select the class with the highest posterior probability.

Assumptions:

- Independence among features; each feature is assumed to contribute independently to the probability.

When to use:

- Text classification tasks (e.g., sentiment analysis, spam filtering).
- Datasets with high dimensionality compared to the number of samples.
- Scenarios requiring fast training and low computational resources.

Multi-class?

multinomial
naive bayes

Bayes' Theorem Formula:

$$P(A|B) = (P(B|A) * P(A)) / P(B)$$

SKLEARN IMPLEMENTATION

1) Import Libraries

```
import numpy as np

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.naive_bayes import MultinomialNB

from sklearn.model_selection import train_test_split
```

2) Split Data

```
notes = [ "patient has chest pain and shortness of breath", "routine checkup no symptoms"]

labels = np.array([1, 0])

notes_train, notes_test, labels_train, labels_test = train_test_split(notes, labels,
test_size=0.3, random_state=29)

vec = CountVectorizer()

clf = MultinomialNB()
```

3) Train/ Test

```
feats_train = vec.fit_transform(notes_train)

feats_test = vec.transform(notes_test)

clf.fit(feats_train, labels_train)

print(f'err: {1 - clf.score(feats_test, labels_test):.4f}')
```